

Declarative Paradigm: Logic Languages

Programming Languages Intro

Dr. Igor Steinmacher

Logic Programming

- Specify **relations** between objects
 - Larger(3, 2)
 - Father(John, Yuri)
- Separate logic from control
 - The programmer defines which **facts** and **relations** are true
 - The system use the facts to solve problems
 - Queries answered based on the facts and relations
- Computational engine: theorem proving and recursion

Prolog!

- Download here:

<https://www.swi-prolog.org/download/stable>

- Online here:

<https://swish.swi-prolog.org/>

Prolog!

- Variables begin with a capital letter
 - Person, Sibling
- Constants are either
 - Integers: 24
 - Atoms: john, 'Person', and '.'
- Structures are predicates with arguments:
 - male(john), parent(john,alice)
- The arity of a structure is its number of arguments (1 and 2 for these ones)

Prolog!

- Variables begin with a capital letter
 - Person, Sibling
- Constants are either
 - Integers: 24
 - Atoms: john, 'Person', and '.'
- Structures are predicates with arguments:
 - male(john), parent(john,alice)
- The arity of a structure is its number of arguments (1 and 2 for these ones)

Logic Programming

- Facts:

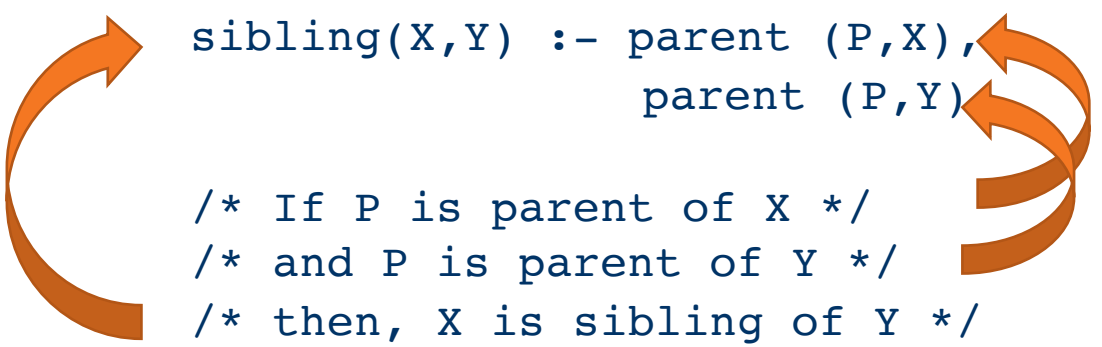
<code>male(john).</code>	<code>parent (john,dante).</code>
<code>female(alice).</code>	<code>parent (ana,dante).</code>
<code>male(dante).</code>	<code>parent (john,alice).</code>
<code>female(ana).</code>	<code>parent (ana,alice).</code>

- We can query (see unification on Person variable):

<code>?- male(john).</code> True	<code>?- parent(ana,john).</code> False
<code>?- male(ana).</code> False	<code>?- parent(Person,alice).</code> Person=john
<code>?- male(Person).</code> Person=john Person=dante	Person=ana

Logic Programming

- Rules:



```
sibling(X,Y) :- parent (P,X),  
                parent (P,Y)
```

```
/* If P is parent of X */  
/* and P is parent of Y */  
/* then, X is sibling of Y */
```

```
parent (john,dante).  
parent (ana,dante).  
parent (john,alice).  
parent (ana,alice).
```

- We can query:

```
?- sibling(alice,dante).  ?- sibling(ana,john).  
    True                      False
```

```
?- sibling(Sib,alice).  
    Sib=alice  
    Sib=dante
```

Some more fun...

(File available in our course)

```
parent(peggy, amy).
parent(peggy, paige).
parent(peggy, paula).
parent(peggy, tim).

parent(woody, amy).
parent(woody, paige).
parent(woody, paula).
parent(woody, tim).

parent(rose, dianne).
parent(rose, dan).
parent(rose, jack).
parent(rose, paul).

parent(george, dianne).
parent(george, dan).
parent(george, jack).
parent(george, paul).

parent(amy, ginger).
parent(amy, jackie).
parent(amy, andrew).
parent(amy, alex).

parent(jack, ginger).
parent(jack, jackie).
parent(jack, andrew).
parent(jack, alex).

parent(ginger, katelyn).
parent(zack, katelyn).

parent(tim, carol).

parent(carol, ashley).
parent(carol, chelsea).

male(jack).
male(andrew).
male(alex).
male(woody).
male(tim).
male(george).
male(dan).
male(paul).
male(zack).

female(amy).
female(ginger).
female(jackie).
female(peggy).
female(paige).
female(paula).
female(rose).
female(dianne).
female(katelyn).
female(carol).
female(ashley).
female(chelsea).
```



```
grandparent(X, Y) :-  
    parent(X, Z),  
    parent(Z, Y).  
  
grandchild(X, Y) :-  
    grandparent(Y, X).  
  
grandfather(X, Y) :-  
    grandparent(X, Y),  
    male(X).
```

```
grandmother(X, Y) :-  
    grandparent(X, Y),  
    female(X).
```

```
mother(X, Y) :-  
    parent(X, Y),  
    female(X).
```

```
father(X, Y) :-  
    parent(X, Y),  
    male(X).
```

```
child(X, Y) :-  
    parent(Y, X).
```

```
partner(X, Y) :-  
    child(Z, X),  
    child(Z, Y),  
    X \= Y.
```

```
son(X, Y) :-  
    child(X, Y),  
    male(X).
```

```
daughter(X, Y) :-  
    child(X, Y),  
    female(X).
```

```
granddaughter(X, Y) :-  
    grandchild(X, Y),  
    female(X).
```

```
grandson(X, Y) :-  
    grandchild(X, Y),  
    male(X).
```

```
sibling(X, Y) :-  
    parent(Z, X),  
    parent(Z, Y),  
    X \= Y.
```

```
sister(X, Y) :-  
    sibling(X, Y),  
    female(X),  
    X \= Y.
```

```
brother(X, Y) :-  
    sibling(X, Y),  
    male(X),  
    X \= Y.
```

```
uncle(X, Y) :-  
    brother(X, Z),  
    child(Y, Z).
```

```
aunt(X, Y) :-  
    sister(X, Z),  
    child(Y, Z).
```

- https://swish.swi-prolog.org/p/family1_nb_Igor.swinb
- https://swish.swi-prolog.org/p/family2_nb_Igor.swinb

What are all pairs grandfather – grandchild?

grandfather(GrandPa,GrandChild)

GrandChild = ginger,
GrandPa = woody
GrandChild = jackie,
GrandPa = woody
GrandChild = andrew,
GrandPa = woody
GrandChild = alex,
GrandPa = woody
GrandChild = carol,
GrandPa = woody
GrandChild = ginger,
GrandPa = george

GrandChild = jackie,
GrandPa = george
GrandChild = andrew,
GrandPa = george
GrandChild = alex,
GrandPa = george
GrandChild = katelyn,
GrandPa = jack
GrandChild = ashley,
GrandPa = tim
GrandChild = chelsea,
GrandPa = tim

Factorial

```
factorial(0, 1). /* Fact: factorial of 0 is 1*/
```



```
factorial(N, Result) :-
```

```
    M is N - 1,          /* If variable M is assigned to N-1 */
```

```
    factorial(M, SubRes), /* and if the factorial of M is SubRes */
```

```
    Result is N * SubRes. /* and if Result is assigned to N*SubRes */
```

```
/* then, the factorial of N is result */
```

https://swish.swi-prolog.org/p/factorial_lgor.swinb

**See you in our next
Lesson!**